HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Office for International Study Programs

Introduction to System on Chip
CO3143

Lab Report

Week 2: Carry Look-ahead Adder (CLA) & RISC-V Assembly

Lecturer: Phạm Kiều Nhật Anh
Class: CC02
Student: Nguyễn Viết Tấn Vương - 2353345

Ho Chi Minh City, 2025

Contents

1 Exercise 1: Carry Look-ahead Adder (CLA) 2
1.1 Theoretical Background 2
1.2 System Design and Implementation 2
1.3 Waveform Analysis 3

2 Exercise 2: RISC-V Assembly 3
2.1 Algorithm and Implementation 3
2.2 Execution Results (Spike Simulator) 4

1 Exercise 1: Carry Look-ahead Adder (CLA)

1.1 Theoretical Background

The Carry Look-ahead Adder (CLA) is designed to overcome the propagation delay inherent in Ripple Carry Adders. Instead of waiting for the carry bit to ripple through each full adder sequentially, the CLA computes the carry signals for higher bits in parallel based on the input bits.
The logic relies on two auxiliary signals:

- **Generate (G):** A carry is generated if both inputs are 1 ($G = A \ \& \ B$).
- **Propagate (P):** A carry is propagated if at least one input is 1 ($P = A \mid B$).

The carry-out is calculated as: $C_{i+1} = G_i \mid (P_i \ \& \ C_i)$.

1.2 System Design and Implementation

The design follows a hierarchical structure to build a 32-bit adder:

1. `gp1`**:** Computes bit-wise generate and propagate signals.
2. `gp4`**:** Computes group G/P signals and internal carries for a 4-bit block using look-ahead logic.
3. `gp8`**:** Combines two `gp4` modules to handle 8 bits.
4. `cla` **(Top-level):** Combines four `gp8` modules to form the 32-bit adder.

```
module gp4(input wire [3:0] gin, pin, input wire cin,
           output wire gout, pout, output wire [2:0] cout);
   assign cout[0] = gin[0] | (pin[0] & cin);
   assign cout[1] = gin[1] | (pin[1] & gin[0]) | (pin[1] & pin[0] & cin);
   assign cout[2] = gin[2] | (pin[2] & gin[1]) |
                    (pin[2] & pin[1] & gin[0]) | (pin[2] & pin[1] & pin[0] & cin);
   assign pout = pin[3] & pin[2] & pin[1] & pin[0];
   assign gout = gin[3] | (pin[3] & gin[2]) |
                 (pin[3] & pin[2] & gin[1]) | (pin[3] & pin[2] & pin[1] & gin[0]);
endmodule
```

Listing 1: gp4 Module Implementation (cla.v)

In the top-level `cla` module, the final sum is calculated using the formula $Sum = A \oplus B \oplus C_{in}$. In the provided code, this is implemented as:

```
// (p & ~g) is equivalent to (A ^ B)
assign sum[i*8] = (p[i*8] & ~g[i*8]) ^ C_blk[i];
```

1.3 Waveform Analysis

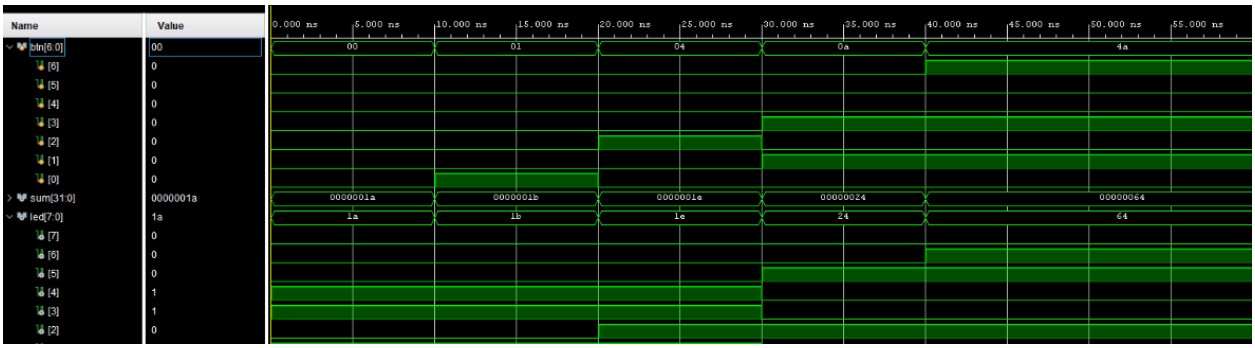

Figure 1: Simulation Waveform of the 32-bit CLA

The simulation results in Figure 1 verify the correctness of the CLA. The testbench (from `system.v`) fixes input A to 26 ($1A_{16}$) and varies input B using button inputs.

- **0 - 5ns:** Input $B = 0$. Sum $= 26 + 0 = 26$ ($1A_{16}$).
- **5 - 20ns:** Input $B = 1$. Sum $= 26 + 1 = 27$ ($1B_{16}$).
- **20 - 30ns:** Input $B = 4$. Sum $= 26 + 4 = 30$ ($1E_{16}$).
- **30 - 40ns:** Input $B = 10$ ($0A_{16}$). Sum $= 26 + 10 = 36$ (24_{16}).
- $>$ **40ns:** Input $B = 74$ ($4A_{16}$). Sum $= 26 + 74 = 100$ (64_{16}).

The immediate update of the ‘sum‘ signal upon input changes confirms the combinational logic is functioning correctly.

2 Exercise 2: RISC-V Assembly

2.1 Algorithm and Implementation

The objective is to write a RISC-V assembly program that converts a NUL-terminated string to uppercase in-place. The algorithm iterates through the string byte by byte:

1. Load a byte from memory.
2. Check if the byte is NUL (0); if so, terminate.
3. Check if the byte is a lowercase letter ('a' $\leq$ char $\leq$ 'z').
4. If lowercase, subtract 32 to convert it to uppercase (since ASCII 'a'=97 and 'A'=65).
5. Store the modified byte back to memory.

```
loop:
    lb t1, 0(t0)            # Load byte
    beq t1, zero, end_loop # Check for NULL terminator

    # Check bounds for lowercase 'a' (97) and 'z' (122)
```

```
    li t2, 97
    blt t1, t2, next_char
    li t2, 122
    bgt t1, t2, next_char

    # Convert to uppercase
    addi t1, t1, -32       # Subtract 32
    sb t1, 0(t0)           # Store back to memory

next_char:
    addi t0, t0, 1         # Increment pointer
    j loop
```

Listing 2: Uppercase Conversion Logic (uppercase.S)

2.2 Execution Results (Spike Simulator)

```
    vuong@vuong:~/SoCs/RISC-V_Assembly_Resource-20251211$ nano link.ld
vuong@vuong:~/SoCs/RISC-V_Assembly_Resource-20251211$ nano Makefile
vuong@vuong:~/SoCs/RISC-V_Assembly_Resource-20251211$ rm uppercase.bin uppercase.o
vuong@vuong:~/SoCs/RISC-V_Assembly_Resource-20251211$ make build
riscv64-unknown-elf-as -march=rv32im -mabi=ilp32 uppercase.S -o uppercase.o
riscv64-unknown-elf-ld -A rv32 --oformat elf32-littleriscv -T link.ld uppercase.o
    -o uppercase.bin
riscv64-unknown-elf-objdump -d uppercase.bin > uppercase.disasm.s
vuong@vuong:~/SoCs/RISC-V_Assembly_Resource-20251211$ make start
spike -p1 -m0x2000:0x1000,0x10000:0x1000 --isa=rv32im -d uppercase.bin
(spike) str 0 2000
Hello world!
(spike) rs 1000
:
(spike) str 0 2000
HELLO WORLD!
(spike)
```

Listing 3: Assembly Program on Spike

The program was compiled and simulated using the `spike` RISC-V simulator (Figure **??**).

1. **Initialization:** The input string "Hello world!" is stored at memory address `0x2000`.
2. **Before Execution:** The command `str 0 2000` displays the string in memory as "Hello world!".
3. **Execution:** The command `rs 1000` runs the processor starting from the program entry point.
4. **After Execution:** Running `str 0 2000` again shows the memory content has changed to "HELLO WORLD!".

This confirms that the logic correctly identified lowercase characters ('e', 'l', 'o', 'w', 'r', 'd') and converted them, while leaving uppercase characters ('H') and punctuation (' ', '!') unchanged.

— End —